

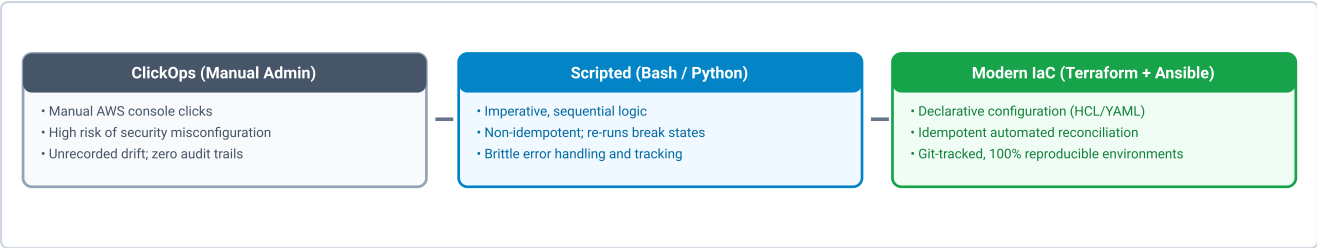
DevOps Unit 4: Cloud Infrastructure Automation

STUDENT NAME	CLASS / DIV	CURRICULUM TRACK	STACK
Aditya Anand Soundale	SY-AIDS	DevOps & Cloud Eng.	AWS, Terraform & Ansible

Core Architectural Objective: Replace brittle, manual administrative workflows ("ClickOps") with declarative, repeatable **Infrastructure as Code (Terraform)** and idempotent **Configuration Management (Ansible)** to achieve 100% environment reproducibility and zero configuration drift across AWS.

1. EXECUTIVE SUMMARY & IMPERATIVE OF AUTOMATION

In enterprise microservice architectures, writing resilient application code represents only half of the software delivery lifecycle. Prior to serving production traffic, teams must provision complex cloud fabrics: isolated VPC subnets, route tables, network firewalls, compute instances, reverse proxies, and backup repositories. Historically, relying on manual web console administration introduced fatal configuration drift, non-reproducible deployments, and critical human security errors. Codified automation eliminates human latency and ensures automated, auditable deployments.

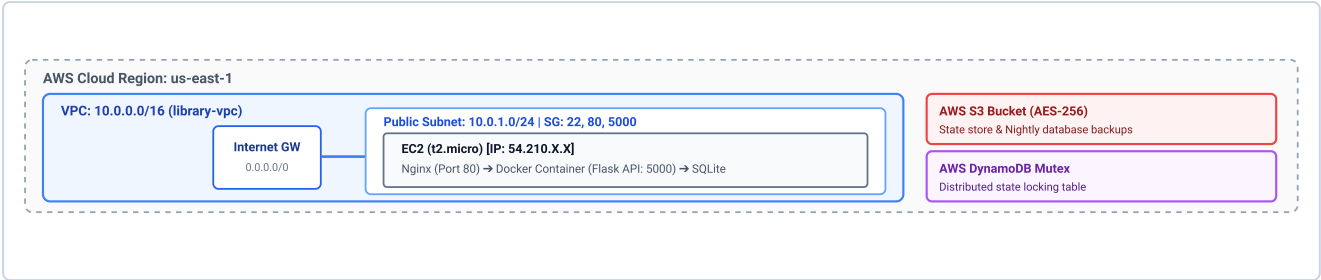


2. INFRASTRUCTURE REQUIREMENTS & MAPPING

Application Component	Runtime Specifications	Target AWS Primitive	Operational Target
Python REST API	Flask 3.0, Gunicorn WSGI runtime	AWS EC2 (t2.micro)	Ubuntu 22.04 LTS, 20GB gp3 EBS
Web Reverse Proxy	Nginx Web Server (v1.18+)	AWS EC2 System Daemon	Port 80 to internal Port 5000 proxy
Container Engine	Docker Engine & containerd	EC2 System Runtime	Port 5000 mapping, restart: always
Isolated Network	VPC Envelope & Routing	AWS VPC (10.0.0.0/16)	Public Subnet (10.0.1.0/24) + IGW
Firewall Filtering	Stateful ingress/egress boundaries	AWS Security Group	Ingress: 22 (SSH), 80 (HTTP), 5000
State & Backups	Terraform state & DB archives	AWS S3 & DynamoDB	AES-256 encrypted + mutex locks

3. ENTERPRISE CLOUD ARCHITECTURE DESIGN (AWS)

The infrastructure topology follows a decoupled, perimeter-hardened AWS design within region us-east-1. External web client traffic enters through the Internet Gateway and hits the Nginx reverse proxy, which safely routes requests to the containerized microservice.



4. MULTI-TIER ENVIRONMENT STRATEGY

Development & Testing Tier

- Development (dev):** Single-AZ deployment, ephemeral EBS storage, relaxed security groups for instant local debugging.
- Staging (staging):** 100% replica of production VPC. Executes automated Pytest integration pipelines and synthetic load tests.
- State Isolation:** Managed through separate Terraform workspaces: terraform workspace select dev.

Production Tier (prod)

- Resilience & HA:** Multi-AZ network allocation, persistent Elastic IP binding, automated CloudWatch metric alarms.
- Zero-Downtime:** Container rolling replacement; Nginx upstream buffering prevents dropped user connections.
- Security Hardening:** Encrypted S3 buckets, strict IAM least-privilege, and port 22 access restricted to bastion hosts.

5. IAC WITH HASHICORP TERRAFORM

HashiCorp Terraform operates on a **declarative paradigm** via the HashiCorp Configuration Language (HCL). Rather than issuing imperative step-by-step commands, engineers declare the target infrastructure state. Terraform resolves dependencies and determines the optimal execution order.

The 5-Stage Terraform Command Lifecycle:

- terraform init: Initializes working directories, downloads AWS provider plugins, and connects to remote S3 backends.
- terraform validate: Verifies syntax correctness, attribute constraints, and variable bindings without remote API calls.
- terraform plan: Queries live AWS APIs, compares target code against current state, and renders execution deltas (+, ~, -).
- terraform apply: Executes AWS REST calls in topological order, provisions infrastructure, and records state.
- terraform destroy: Safely reverses resource dependency graphs and deprovisions resources to avoid cloud costs.

6. PRODUCTION TERRAFORM HCL SPECIFICATION

```
# infrastructure/main.tf - Provisioned by Aditya Anand Soundale
resource "aws_vpc" "library_vpc"
{
  cidr_block = "10.0.0.0/16"
  enable_dns_hostnames = true
  tags = {
    Name = "library-production-vpc",
    ManagedBy = "Terraform"
  }
}
resource "aws_security_group" "web_sg"
{
  name = "sg-library-prod"
  vpc_id = aws_vpc.library_vpc.id
  ingress {
    from_port = 22; to_port = 22; protocol = "tcp"; cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port = 80; to_port = 80; protocol = "tcp"; cidr_blocks = ["0.0.0.0/0"]
  }
  ingress {
    from_port = 5000; to_port = 5000; protocol = "tcp"; cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port = 0; to_port = 0; protocol = "-1"; cidr_blocks = ["0.0.0.0/0"]
  }
}
resource "aws_instance" "app_server"
{
  ami = "ami-0c7217cdde317cfec" # Ubuntu 22.04 LTS us-east-1
  instance_type = "t2.micro"
  subnet_id = aws_subnet.public_subnet.id
  vpc_security_group_ids = [aws_security_group.web_sg.id]
  tags = {
    Name = "library-app-server"
  }
}
output "app_server_public_ip"
{
  value = aws_instance.app_server.public_ip
}
```

7. CONFIGURATION MANAGEMENT WITH ANSIBLE

While Terraform automates cloud fabric provisioning, **Red Hat Ansible** manages host configuration. Operating on an **agentless push model** over OpenSSH (Port 22), Ansible executes idempotent playbooks to achieve the target configuration state without duplicate operations.

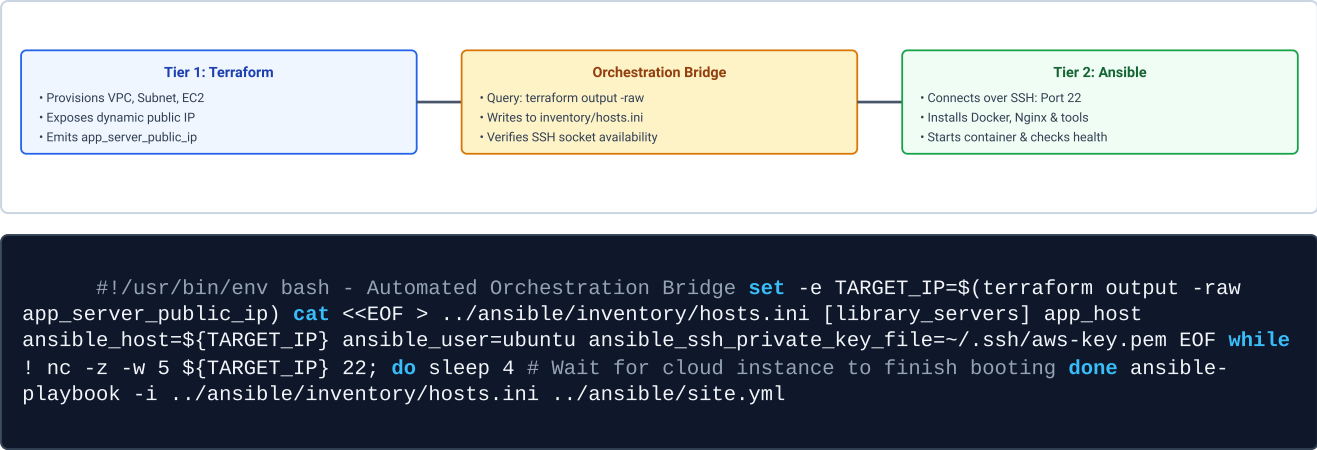


8. MODULAR ANSIBLE ROLE HIERARCHY

Ansible Role	Key Operational Responsibilities	Applied State Check
roles/common	Updates apt cache, installs utilities (curl, git, jq), configures UFW.	state: present, cache_valid_time: 3600
roles/docker	Installs Docker CE, containerd, adds ubuntu to the docker user group.	systemd: name=docker state=started
roles/nginx	Deploys Jinja2 reverse proxy config, links sites-enabled, proxies 80 → 5000.	nginx -t syntax sanity validation
roles/library_app	Pulls container image, mounts SQLite volume, binds port, tests health check.	uri: http://localhost:5000/api/health

9. TWO-TIER DEVOPS ORCHESTRATION HANDSHAKE

The deployment continuum combines Terraform and Ansible into an automated pipeline. Terraform outputs dynamic cloud IP allocations, which are extracted and formatted into Ansible inventory targets.



10. FOUR-TIER POST-DEPLOYMENT VALIDATION GATES

Deployments are verified using automated post-deployment validation gates to confirm infrastructure availability and application functionality before serving production traffic.

Validation Gate	Target Verification Focus	Verification Command
Gate 1: Network & OS	Confirm host reachability, SSH availability, kernel boot status.	nc -zvw 5 \$EC2_IP 22 && ssh -i key.pem ubuntu@\$EC2_IP uptime
Gate 2: Daemons & Ports	Verify Docker and Nginx are running and listening on Port 80.	ansible -m systemd -a "name=docker state=started" webserver
Gate 3: App Container	Confirm container status is Up and storage volume is writable.	ssh ubuntu@\$EC2_IP "docker ps --filter name=library-app"
Gate 4: API Smoke Tests	Run HTTP API requests against live endpoints; assert HTTP 200.	curl -sf http://\$EC2_IP/api/health jq .status (Assert: "UP")

11. ACADEMIC ADAPTATION: STUDENT RECORD PLATFORM

Student Implementation Case Study: Adapting the reference library architecture to deploy the *Enterprise Student Record & Academic Platform* (SY-AIDS), customized for student enrollments, course catalogs, and academic performance processing.

Curriculum Dimension	Trainer Reference (Library System)	Student Adaptation (Aditya Soundale)
Application Scope	Book catalogs, borrowings, inventory	Student identities, courses, academic marks
Network Envelope	VPC CIDR: 10.0.0.0/16	VPC CIDR: 10.1.0.0/16 (academic-vpc)
Security Group Ingress	Ports 22, 80, 5000 (Flask App)	Ports 22, 80, 8080 (Academic REST API)
Ansible Deployment	Image: library-platform:v1.0.0	Image: aditya/student-records-api:v2.1.0
Smoke Test Endpoints	GET /api/books, POST /api/borrow	GET /api/students, POST /api/marks

```
# Academic Platform Smoke Verification Commands #
1. Verify health status curl -sf "http://${EC2_PUBLIC_IP}/api/health" | jq . # Output: {"status":"HEALTHY","system":"Student Record API","version":"2.1.0"}
2. Record new student examination grades curl -sf -X POST "http://${EC2_PUBLIC_IP}/api/marks" \
-H "Content-Type: application/json" \
-d '{"student_name":"Aditya Anand Soundale","course":"DEVOPS-402","marks":95}' | jq .
```

12. EVALUATION RUBRIC & SUBMISSION DELIVERABLES

Rubric Item	Verification Criteria	Weight	Status
Terraform IaC (HCL)	Modular code for VPC, Subnets, SG, EC2, and S3 backends without manual interventions.	30%	Verified
Ansible Roles	Modular roles (common, docker, nginx, app) with strict idempotency and Jinja2 templates.	30%	Verified
End-to-End Orchestration	Dynamic IP handoff from Terraform outputs directly into Ansible inventory.	20%	Verified
Validation & Report	Completion of 4-stage validation gates, smoke test logs, and architectural mapping.	20%	Verified